

Il Libro del Corso

Corso di Informatica — tutti i documenti in uno

Versione 1.3 · 16/08/2026

Corso a cura del prof. Nicola Regge

Indice

CLASSE 1 — INFORMATICA

Programma del Corso v0.3

Che corso è (e che corso NON è)
Una scatola flessibile, cucita sui ragazzi
A chi è rivolto e con che spirito
Il vincolo pratico della scuola: niente installazioni (dove si può)
I moduli dell'anno
Il filo dell'anno (sequenza e periodi indicativi)
Dove porta: il percorso pluriennale
Serbatoio di idee extra (competenze spendibili nel lavoro)
Come si valuta
Uso dell'AI
Prossimi passi (roadmap del programma)
Storia delle versioni (le modifiche fatte)

La Bussola del Lavoro v0.1

La verità di partenza (chi assume a 15-17 anni)
Cassetto 1 — La testa e il cuore (quello che pesa di più)
Cassetto 2 — Le mani (le competenze tecniche che si "vendono" subito)
Cassetto 3 — Le carte (i documenti che fanno la differenza)
La sintesi
Ingredienti da dosare (come si usa questa bussola)
Storia delle versioni (per noi)

Da Far Fare Assolutamente v0.1

1. Toccare un database vero e scrivere un po' di SQL
2. Costruire uno shop e-commerce funzionante (demo)

Storia delle versioni (per noi)

Il Mio Negozio Online — Guida per i ragazzi v1.4

TAPPA 1 — Metti il negozio ONLINE (la prima vittoria) 🌐
TAPPA 2 — Collega il database della classe 🗄️
TAPPA 3 — Ricevi gli ordini via email ✉️
TAPPA 4 — Fallo tuo 🍌
La prova del nove 🧠
Se qualcosa non va 🛠️ (succede a tutti)

Il Mio Negozio Online — Piano-lezione v1.0

In breve

Prima di iniziare — cosa prepara il prof (una volta sola)

La scaletta (3 lezioni)

Canovaccio — spiegare il database dal vivo (10-15 min)

Se lavori a gruppi (opzionale, 2-4 ragazzi)

Gestire i ritmi diversi (importante per questa classe)

Valutazione (coerente col corso)

Checklist da tenere in aula

CORSO GODOT / GDSCRIPT

Il Manuale di Godot v0.5

Scheda 1 — Come si valutano i compiti

Scheda 2 — Scrivere in Markdown

Capitolo 0 — Cos'è Godot, il parente di Lazarus

Capitolo 1 — I 4 concetti base di Godot

Capitolo 2 — GDScript: il linguaggio

Capitolo 3 — Il nostro primo gioco: "Chirurgo Pasticcione"

Capitolo 4 — Il percorso: dagli esercizi al "progetto boss"

Come useremo l'AI

Changelog del manuale

Eserciziario di Godot v0.5

Come funziona ogni esercizio

Esercizio 1 — Il bottone che saluta

Esercizio 2 — Muovi il quadrato

Esercizio 3 — Prendi la moneta

Esercizio BOSS — Affonda la Bonomi

Changelog dell'eserciziario

Quaderno dello Studente (modello)

Come si usa (semplice)

PAGINA — Lezione (copia questo blocco ogni volta)

PAGINA — Il mio gioco (copia questo blocco per ogni esercizio)

Programma del Corso

Versione 0.3 · 26/07/2026 · Parte: Classe 1 — Informatica

Che corso è (e che corso NON è)

Questo **non** è un corso di “informatica di base” fatto di clic semplici: quelle cose i ragazzi le vedono altrove. Qui facciamo **tutta l’informatica**, con un taglio **più tecnico e ambizioso** — da futuri **tecnici**, non da semplici utenti. Vogliamo che capiscano **come funzionano davvero** le cose: cosa succede dentro un programma, come viaggiano i dati in rete, cosa c’è dentro un computer e come si configura, monta e mette in funzione.

Il trucco per non spaventarli: contenuti più avanzati, **stesso metodo** — piccoli passi, una vittoria concreta a ogni tappa, tanto pratico, poca teoria e solo quella che serve al fare. “Avanzato” qui vuol dire **profondo e vero**, non “difficile e respingente”.

Una scatola flessibile, cucita sui ragazzi

I sei moduli **non** sono un percorso rigido da percorrere tutto uguale per tutti. Sono **contenitori** — delle **manopole** che il docente apre o chiude a seconda di **come la classe lo segue**. Il programma è la scatola; il vestito lo cuciamo addosso ai ragazzi che abbiamo davanti.

- **Classe in difficoltà** → si spinge di più sulla **G Suite** e sulle competenze **subito spendibili**: cose concrete che aprono lavoro anche a chi fa più fatica.
- **Classe che va forte** → si va lunghi su **Lazarus** e sulla programmazione, fino ad arrivare (negli anni) a **configurare reti complesse**.
- In mezzo, ogni sfumatura: si allunga un modulo, se ne accorcia un altro, si anticipa o si rimanda. Nessun modulo è “obbligatorio nella sua interezza”.

Il metro di ogni scelta è uno solo: dare a **questi** ragazzi il **massimo delle chance** di trovare un lavoro e di essere **valutati nel mondo del lavoro**. Non “finire il programma”: **spendere bene** il tempo che abbiamo con loro.

***Il senso di tutto.** Sono quasi tutti extracomunitari e ragazze in difficoltà, “scarti” di altre scuole. Per loro questo corso vuole essere una **nuova spiaggia**: un posto dove si riparte, si viene trattati con dignità e si esce con qualcosa di vero in mano. La flessibilità della scatola serve esattamente a questo — non lasciare indietro nessuno e portare ognuno il più lontano che può.*

A chi è rivolto e con che spirito

Siamo in una **prima** di **istituto professionale**. Molti ragazzi arrivano da percorsi difficili, alcuni con un background migratorio, alcuni già “scartati” da altre scuole. Qui **non sono scarti**: gli diamo un corso fatto bene, con dignità, pensato per **aprire porte** e portare a **sbocchi lavorativi migliori**.

Le regole che valgono per ogni modulo:

- **Vinci subito (≈10 min)**: la prima cosa che fanno deve funzionare quasi da sola. La prima vittoria è ciò che impedisce la fuga.
- **Fallo tuo**: in ogni attività scelgono qualcosa di **loro** — il budget, il PC dei sogni, il colore, il nome, la foto.
- **Mostralo**: ogni pezzo dev’essere **mostrabile** — al compagno, sul telefono, a casa.
- **Errore = zero vergogna**: annullare è facile, sbagliare è normale (“capita a tutti, anche ai professionisti”).
- **La prova del nove — “saperlo spiegare”**: se sanno raccontare a voce, con parole loro, cosa hanno fatto, la competenza c’è davvero.

Il vincolo pratico della scuola: niente installazioni (dove si può)

A scuola installare software richiede l'amministratore di sistema (lento). Quindi prediligiamo tutto ciò che gira **da browser** o è **portabile** — con l'eccezione, voluta, dei moduli in cui **mettere le mani è il punto** (montaggio del PC, installazione del sistema operativo): lì si lavora su **macchine di laboratorio/di recupero**, non sui PC “buoni” della scuola.

Modulo	Come lo facciamo
Software / editor / compilatore	Editor da browser o portabili; poi Lazarus (portabile) per “vedere” un compilatore vero.
Reti e apparati di casa	Teoria + osservazione di apparati veri (router, switch); simulatore da browser dove serve.
Configurazione PC su Amazon	Tutto da browser : catalogo Amazon + un Foglio Google per la lista componenti.
Montaggio fisico + sistema operativo	Su PC di laboratorio/recupero , non sui PC di produzione della scuola.
G Suite (lato tecnico)	Tutto da browser .
Lazarus	Versione portabile (una cartella, doppio clic); ripiego: Pascal online.

I moduli dell'anno

Sei moduli. I primi cinque costruiscono il “tecnico”; **Lazarus parte a metà anno (o prima)** e corre fino a fine anno, in parallelo agli ultimi moduli, come ponte verso il secondo anno.

Modulo 1 — Cos'è l'informatica davvero: software, editor, compilatore

Apriamo il cofano: cosa c'è dietro un “programma”, una parola alla volta.

Perché per primo: dà le **parole giuste** e il modello mentale che regge tutto l'anno. È il modulo che segna la differenza tra “so usare il PC” e “**so come funziona**”.

Cosa impariamo:

- **Hardware e software:** la differenza, ma andando a fondo — cos'è davvero un programma (istruzioni che la macchina esegue una dopo l'altra).
- **Cos'è un editor:** il posto dove si **scrive** (testo o codice). Non fa girare niente: scrive e basta.
- **Cos'è un compilatore:** il **traduttore** che prende quello che scrivo io (il “sorgente”) e lo trasforma in un **programma che il computer sa eseguire**. La catena: *scrivo nell'editor → il compilatore traduce → nasce l'eseguibile*.
- **Linguaggi di programmazione** in due parole: perché ne esistono tanti.
- **Dati, bit e byte:** come si misura la roba digitale (KB/MB/GB), quanto “pesa” una foto o un video.

Prima vittoria (≈10 min): scrivere due righe in un editor e **vedere con i propri occhi** la differenza tra un file di testo e un programma che gira.

Ponte: questo modulo prepara **Lazarus** — lì il compilatore lo useranno per davvero, e capiranno *cosa* sta succedendo quando premono “Esegui”.

Modulo 2 — Le reti: come viaggiano i dati

Quando mando un messaggio, cosa succede davvero? Seguiamo il pacchetto.

Perché qui: le reti sono il cuore del percorso pluriennale (arriveranno a **Cisco Packet Tracer** negli anni successivi). Qui si mettono le fondamenta, in modo concreto e visibile.

Cosa impariamo:

- **Cos'è una rete:** computer che si parlano.

- **I pacchetti:** i dati non viaggiano “interi”, si spezzano in **pacchetti** che partono, viaggiano e si **ricompongono** all’arrivo. L’idea chiave di Internet.
- **Gli indirizzi (IP)** spiegati semplice: ogni dispositivo ha un “numero di casa” in rete.
- **Gli apparati di rete che hai in casa**, cosa fa ciascuno:
- **Modem** — la porta verso Internet (il “cancello di casa”).
- **Router** — smista il traffico tra i dispositivi e Internet (il “vigile”).
- **Switch** — collega più dispositivi via cavo (la “ciabatta intelligente”).
- **Access Point / Wi-Fi** — la rete senza fili.
- **Internet come “rete di reti”.**

Prima vittoria (≈10 min): disegnare la rete di casa propria — chi è il modem, chi il router, cosa è attaccato via cavo e cosa via Wi-Fi. Poi guardare un apparato **vero** e riconoscerne le porte.

Ponte: è il primo passo verso Cisco Packet Tracer (2°–4° anno).

Modulo 3 — L’hardware e la configurazione del PC (con Amazon e un budget)

Ti do 700 euro: mettimi insieme il PC migliore possibile. Come un vero tecnico.

Perché qui: è il modulo che li accende. Trasforma una noiosa lista di componenti in una **caccia al tesoro con un budget** — realistica, utile, e con cifre vere che vedono ogni giorno.

Cosa impariamo:

- **I componenti, uno per uno, e cosa conta di ciascuno:** scheda madre, **CPU** (il processore), **RAM**, **SSD/HDD** (memoria che resta), **scheda video** (GPU), **alimentatore**, case, raffreddamento.
- **Come leggere una specifica:** cosa vogliono dire i numeri (GHz, GB, TB...).
- **La compatibilità di base:** perché non tutti i pezzi vanno insieme (es. il **socket** della CPU e la scheda madre).
- **Configurare un PC su Amazon dentro un budget:** dato un tetto di spesa (es. **500 • 800 • 1200 €**), scegliere i componenti giusti e compatibili, e motivare le scelte.

Prima vittoria (≈10 min): “il mio PC entro il budget” — una lista dei componenti con prezzi, messa in un **Foglio Google** che fa la **somma** e dice se sfori o no.

Progetto mostrabile: a gruppi, **la miglior build a parità di budget** — si confrontano le scelte e si “difende” la propria (prova del nove: sai spiegare perché quella CPU e non un’altra?).

Modulo 4 — Montaggio fisico e sistema operativo

Dalla lista alla macchina vera: la monto, la accendo, ci installo Windows.

Perché qui: dopo aver **scelto** i pezzi (Modulo 3), li **montano** davvero. Il salto dal virtuale al fisico è potentissimo per la motivazione.

Cosa impariamo:

- **Sicurezza prima di tutto:** staccare la corrente, l'elettricità statica (il braccialetto antistatico), come si maneggiano i pezzi.
- **Montare e smontare un PC vero:** dove va ogni componente, i connettori, i versi giusti (niente forza bruta).
- **Il primo avvio e il BIOS/UEFI** in due parole: capire se la macchina “vede” tutto.
- **Installare il sistema operativo da zero:** preparare una **chiavetta avviabile**, installare Windows (o Linux), i primi passi dopo l'installazione.
- **I tool di sistema più importanti:** Gestione attività (Task Manager), Gestione dispositivi, gestione dischi, driver, impostazioni/pannello di controllo, backup. Cosa guardare quando “qualcosa non va”.

Prima vittoria (≈10 min): un PC montato dal gruppo che **si accende** — la foto/video del “momento accensione”. Soddisfazione enorme.

Progetto mostrabile: un PC di recupero **montato e con il sistema operativo installato**, pronto all'uso. Roba da tecnico.

Modulo 5 — G Suite, il lato tecnico (le cose difficili)

Non le basi: le cose che fanno dire “non sapevo si potesse fare”.

Perché qui: le basi della G Suite le vedono con altri. Noi puntiamo alla parte **tecnica e potente**, quella spendibile in un ufficio vero.

Cosa impariamo (selezione, la tariamo strada facendo):

- **Permessi e condivisione fine:** chi può **vedere**, chi **commentare**, chi **modificare**; cartelle condivise; link e loro rischi.
- **Fogli sul serio:** formule vere (**SOMMA**, **SE**, **CERCA**), grafici, filtri — aggancia il “conto” della calcolatrice di Lazarus.
- **Moduli (Form):** creare un quiz/sondaggio che **raccoglie le risposte in automatico** dentro un Foglio.

- **Organizzazione avanzata del Drive** e la **cronologia delle versioni** di un file (tornare indietro nel tempo).

Prima vittoria (≈10 min): un **modulo/quiz** che, appena un compagno risponde, riempie da solo una tabella. Effetto “magia”.

Modulo 6 — Lazarus: la prima programmazione (da metà anno)

Adesso i programmi non li uso: li FACCIO io. Bottoni, finestre, tutto mio.

Perché qui e quando: parte **da metà anno (o prima)** e corre in parallelo fino a giugno. È il coronamento dell’anno e il **ponte diretto al secondo anno** (Godot e Lazarus più avanzati). Aggancia il Modulo 1: qui il **compilatore** che avevano solo sentito nominare lo usano premendo “Esegui”.

Cosa impariamo (piccoli passi):

- **L’ambiente di sviluppo (l’IDE)** e la **Form**, la finestra del programma.
- **I componenti:** **TButton** (bottone), **TEdit** (casella di testo), **TLabel** (scritta), **TMemo** (testo su più righe — per un mini blocco note).
- **Le proprietà:** **Caption**, colori, posizione — si cambiano cliccando.
- **L’evento click:** far succedere qualcosa quando premo un bottone.
- **Variabili e un po’ di logica:** leggo un TEdit, faccio un conto, mostro il risultato in una TLabel.
- **Programmini veri:** una **calcolatrice**, un **mini blocco note** con TMemo, un **quiz a bottoni**.

Prima vittoria (≈10 min): un bottone che, premuto, cambia la scritta: “Ciao, *il mio nome!*”. Il primo programma **mio** che reagisce.

Ponte: componenti → proprietà → eventi qui; in **Godot** (2° anno) diventano **nodi** → **proprietà** → **segnali**. Chi capisce ora, in seconda parte avvantaggiato.

Il filo dell'anno (sequenza e periodi indicativi)

Periodo	In primo piano	In parallelo
Inizio anno	M1 Software/editor/compilatore · M2 Reti	—
Autunno–inverno	M3 Configurazione PC (Amazon + budget)	inizio M5 G Suite tecnica
Metà anno	M4 Montaggio + sistema operativo	inizia M6 Lazarus
Seconda metà	M6 Lazarus (programmini)	completamento M4/M5

Trasversale a tutto l'anno — il quaderno dello studente: ogni ragazzo tiene un **suo** quaderno (in Documenti Google) che cresce a ogni lezione: cosa ho fatto, uno screenshot/foto, cosa ho capito con parole mie. A fine anno è un portfolio **loro** di cui essere fieri — ed è la “prova del nove” del saper spiegare.

Dove porta: il percorso pluriennale

Questo primo anno getta **fondamenta larghe**. Ecco dove conducono, così ogni modulo ha un “perché” grande dietro:

Anno	Cosa si fa
1° (questo)	Fondamenta di tutta l’informatica (software, reti, hardware, sistema operativo, G Suite tecnica) + primo Lazarus da metà anno.
2°	Godot e Lazarus più avanzati (programmazione vera) + introduzione alle reti con Cisco Packet Tracer .
3°	Si rafforza un filone : le reti (Cisco Packet Tracer), la programmazione , oppure l’ assemblaggio/hardware — a seconda del gruppo.
4°	Cisco Packet Tracer avanzato : progettare una rete reale . Traguardo già dimostrato quest’anno: la rete di una scuola su due piani — due aule di informatica più segreteria e amministrazione — con backbone (le dorsali che collegano tutto).

*Il quarto anno non è un sogno: è **già stato fatto**. Questo è ciò che i ragazzi possono davvero raggiungere partendo da qui. Serve a noi (per tenere la rotta) e a loro (per sapere dove stanno andando).*

Serbatoio di idee extra (competenze spendibili nel lavoro)

Cose **alla loro portata** che nel mondo del lavoro pesano, da pescare quando la classe lo permette (in prima, seconda, terza — e in quarta quando la facciamo). Non sono moduli obbligatori: sono **carte in più** da mettere nel loro bagaglio.

Idea	Perché è spendibile	Perché è alla loro portata
Crimpare i cavi di rete (montare i connettori sui cavi con la pinza)	Lavoro vero da tecnico di rete; si vede subito se funziona	Manuale, economico, “figo”; si lega al Modulo 2 (reti) e a Cisco
Riparazione e manutenzione PC (sostituire un pezzo, pulire, reinstallare)	Sbocco concreto: assistenza, negozi, “help desk” (il banco assistenza)	È il naturale seguito del montaggio (Modulo 4)
Digitazione veloce alla tastiera (scrivere senza guardare)	Utile in ogni lavoro d’ufficio; fa risparmiare ore	Si allena da browser, gratis, un po’ per volta
Competenze per il lavoro: scrivere il curriculum, un’email seria, prepararsi a un colloquio	Per questi ragazzi può cambiare le cose davvero	Si fa con la G Suite che già usano (Modulo 1/5)
Una tua pagina web (le basi di HTML e CSS, i “mattoni” dei siti)	Base per tanti lavori digitali; portfolio da mostrare	Risultato visibile subito → motiva (Mostralo)
Linux, primo assaggio (un altro sistema operativo, gratuito)	Molto richiesto nel mondo tecnico e delle reti	Si prova sui PC di laboratorio (Modulo 4)
Sicurezza informatica di base (password, truffe online, copie di sicurezza)	Ogni azienda la chiede; poca teoria, molto buonsenso	Si aggancia a reti (Modulo 2) e sistema operativo (Modulo 4)
Certificazioni (un “patentino” ufficiale: ICDL oggi, Cisco negli anni dopo)	Un pezzo di carta riconosciuto vale nel curriculum	Traguardo a tappe, coerente col percorso pluriennale

*Come sempre: si aggiunge ciò che **serve a loro** e che **riescono a mostrare**. Meglio poche cose fatte bene e spendibili, che tante di fretta.*

Come si valuta

Come nel corso di Godot, le **regole sono chiare fin da subito** e non si basano sul “copiare bene”:

1. **Il lavoro che funziona è il biglietto d’ingresso, non il voto.**
2. **Il voto nasce dalla prova dal vivo:** me lo **spieghi** a parole tue, oppure ti do il tuo lavoro con **un piccolo intoppo** e lo **rimetti a posto** lì per lì.
3. **Il patto con l’AI e con i compagni:** si usano per **imparare**, non per consegnare senza capire.
4. **Zero vergogna:** usare gli aiuti è permesso e normale; sbagliare è normale.

Uso dell'AI

L'AI è come la **calcolatrice in matematica**: aiuta, ma se non capisci cosa stai facendo non serve a niente. Sì per: capire un errore, farsi spiegare, avere uno spunto. No per: farsi fare tutto e consegnarlo senza capirlo. **Prova del nove**: se sai spiegare a voce cosa hai fatto, la competenza c'è.

Prossimi passi (roadmap del programma)

Questo file è la **mappa**. Da qui, un modulo alla volta, produrremo i contenuti veri (in `classe-1/manuale.md` e `classe-1/eserciziario.md`, con lo stesso stile e la stessa impaginazione del corso di Godot):

1. **Modulo 1 — Software, editor, compilatore:** prima scheda “Vinci subito”.
2. A seguire gli altri moduli, con Lazarus (M6) da preparare per metà anno.
3. Adattare il generatore PDF (`manuale/_build/`) per una copertina “Corso di Informatica — Classe 1” (oggi è marcato “Corso di Godot”).

Ogni consegna sarà **versionata** e **congelata** come i manuali di Godot: se cambia il contenuto, si **alza il numero di versione** e si aggiunge una riga alla tabella delle modifiche in fondo.

Storia delle versioni (le modifiche fatte)

Versione	Data	Cosa è cambiato
0.1	26/07/2026	Prima stesura: quattro moduli generici (Suite Google, informatica di base, Lazarus, assemblaggio PC), sequenza dell'anno, valutazione, uso AI.
0.2	26/07/2026	Riscrittura sulla visione di Nicola: taglio tecnico/avanzato (non “informatica di base”). Sei moduli — Software/editor/compilatore · Reti e pacchetti e apparati di casa · Configurazione PC su Amazon con budget · Montaggio fisico + sistema operativo + tool · G Suite lato tecnico · Lazarus da metà anno (bottoni, finestre, Label, Memo). Aggiunto il percorso pluriennale (2°–4° anno) fino a Cisco Packet Tracer e alla rete di scuola del 4° anno.
0.3	26/07/2026	Aggiunta la sezione “Una scatola flessibile ”: i moduli sono manopole che si aprono/chiodono in base alla classe, tutto al servizio delle chance di lavoro (la “nuova spiaggia”). Aggiunto il “ serbatoio di idee extra ” con competenze spendibili alla loro portata (crimpare cavi, riparazione PC, digitazione, curriculum/colloquio, pagina web, Linux, sicurezza, certificazioni). Tolle parole inglesi non spiegate (“bump” → “alzare il numero di versione”; “changelog” → “storia delle versioni”).

La Bussola del Lavoro

Versione 0.1 · 27/07/2026 · Parte: Classe 1 — Informatica

La verità di partenza (chi assume a 15-17 anni)

A questa età, nei tirocini e nel primo lavoro, **quasi nessuno assume per le competenze tecniche**: quelle il datore di lavoro le insegna. Assume per **l'atteggiamento**, e poi controlla se il ragazzo sa fare davvero **due o tre cose concrete e mostrabili**.

Tradotto per il corso: la tecnica è il **biglietto da visita**; ciò che fa dire “questo lo prendo” è soprattutto **come si comporta e cosa sa mostrare**.

Le cose che servono stanno in **tre cassetti**.

Cassetto 1 — La testa e il cuore (quello che pesa di più)

Sono le prime cose che un datore di lavoro guarda in un ragazzo giovane. Per i nostri studenti, che spesso partono da lontano, questo cassetto è il vero riscatto.

- **Affidabilità:** arrivare in orario, tutti i giorni; avvisare se non puoi. Sembra banale ed è la **prima** cosa che fa tenere (o cacciare) un ragazzo in tirocinio.
- **Saper comunicare:** rispondere al telefono, scrivere un'email educata, parlare con un cliente senza sparire. Per chi ha un background migratorio, curare l'**italiano "da lavoro"** apre porte che la sola tecnica non apre.
- **Ammettere un errore invece di nasconderselo.** I responsabili si fidano di chi dice "ho sbagliato qui". È lo stesso "errore = zero vergogna" del nostro metodo.
- **Voglia di imparare e un po' di iniziativa:** non restare fermi ad aspettare l'ordine; provare a capire da soli.
- **Non mollare al primo "no".** Per ragazzi che si arrendono per difesa, allenare la resistenza è una **competenza lavorativa** a tutti gli effetti.
- **Stare in squadra:** chiedere aiuto quando serve, rispettare i compagni e chi coordina.

Questo cassetto il corso lo allena già col metodo — Vinci subito · Fallo tuo · Mostralo e la prova del nove del "saperlo spiegare". È il nostro vantaggio più grande: poche altre scuole glielo danno.

Cassetto 2 — Le mani (le competenze tecniche che si “vendono” subito)

In ordine di **quanto è facile trasformarle in un lavoro** a quell'età:

1. **Assistenza e riparazione PC** — montare, sostituire un pezzo, reinstallare, togliere un virus. Negozi, centri assistenza e “banchi di assistenza” (in inglese *help desk*) assumono ragazzi così.
2. **Cablaggio e apparati di rete** — montare i connettori sui cavi con la pinza (“crimpare”), riconoscere router e switch, configurazioni di base. Gli installatori di reti cercano manodopera giovane.
3. **Fogli di calcolo e strumenti d'ufficio fatti bene** — formule, tabelle, ordine. Lo chiede quasi ogni ufficio.
4. **Installare e gestire i sistemi operativi** — Windows e un assaggio di **Linux** (un altro sistema operativo, gratuito, molto richiesto nel tecnico).
5. **Sicurezza informatica di base** — password robuste, riconoscere le truffe via email (in inglese *phishing*), fare copie di sicurezza. Le aziende ci tengono moltissimo: anche solo il buonsenso vale.
6. **Programmazione e logica** — Lazarus, poi Godot. Più che “fare il programmatore a 16 anni”, serve a **capire** e a **distinguersi**.

Cassetto 3 — Le carte (i documenti che fanno la differenza)

- **Sicurezza sul lavoro:** in Italia, per fare un tirocinio scuola-lavoro (il cosiddetto **PCTO**, cioè i percorsi per le competenze trasversali e l’orientamento) serve il corso sulla sicurezza. Averlo rende un ragazzo **subito collocabile**: è la carta più concreta di tutte.
- **Certificazioni** — un “patentino” riconosciuto: l’**ICDL** (la ex “patente del computer”) prima, le certificazioni **Cisco** sulle reti negli anni successivi. Nel curriculum di un professionista pesano davvero.
- **Un curriculum pulito e saper affrontare un colloquio.** Per i nostri ragazzi può cambiare le cose.
- **La raccolta dei lavori** (in inglese *portfolio*): il **quaderno dello studente** e i progetti mostrabili sono già un mini-portfolio. Un ragazzo che al colloquio apre il telefono e dice “guarda, questo l’ho fatto io” vince su chi ha solo parole.

La sintesi

Per i nostri ragazzi il moltiplicatore **non** è la tecnica avanzata: è **tecnica di base solida + affidabilità + saper comunicare + qualcosa da mostrare**. Un diplomato puntuale, che sa parlare con un cliente, monta un PC, mette su una piccola rete e ha una raccolta di cose fatte è **immediatamente assumibile**. È lì che punta il corso.

Ingredienti da dosare (come si usa questa bussola)

Gli argomenti del corso **non** hanno una divisione fissa decisa a tavolino. Sono **ingredienti** che il docente dosa **in base alla classe che trova**:

- C'è la classe a cui **piace programmare** → si spinge Lazarus/Godot.
- C'è la classe che la **programmazione la detesta** → si va di più su hardware, reti, assistenza.
- C'è la classe che **vuole assemblare** → si parte dalle mani sull'hardware.
- E c'è la classe che dice “voglio assemblare” e poi **non ha voglia di fare niente** → si ricomincia dalle vittorie facili e mostrabili per riagganciarla.

Quindi: **parsimonia e adattamento**. Si usa questa bussola per scegliere *cosa* spingere, momento per momento, senza sensi di colpa se un ingrediente resta nel cassetto. **Il metro è sempre uno solo**: dare a questi ragazzi il massimo delle chance di trovare un lavoro ed essere valutati nel mondo del lavoro. Non “finire il programma”: **spendere bene** il tempo che abbiamo con loro.

Storia delle versioni (per noi)

Versione	Data	Cosa è cambiato
0.1	27/07/2026	Prima stesura della bussola: la verità su chi assume a 15-17 anni, i tre cassette (testa e cuore · le mani · le carte), la sintesi e il principio degli “ingredienti da dosare” in base alla classe.

Da Far Fare Assolutamente

Versione 0.1 · 27/07/2026 · Parte: Classe 1 — Informatica

1. Toccare un database vero e scrivere un po' di SQL

Cosa devono fare: creare qualche tabella, metterci dei dati e scrivere le prime **query SQL** (le “domande” al database, tipo “*dammi tutti i prodotti sotto i 20 euro*”).

Perché è irrinunciabile: il database sta **sotto** quasi ogni sito, negozio online e gestionale. Saperlo toccare è una competenza spendibile subito, e prepara il progetto dello shop (l’elenco dei prodotti è un database).

Con quali strumenti (in ordine, dal più facile al più “da lavoro”):

- **Primo assaggio, zero account:** `sqliteonline.com` — solo browser, in dieci minuti scrivono la prima query. *Vinci subito.*
- **Strumento visuale “da lavoro”:** **phpMyAdmin** + **MariaDB** (via **XAMPP portable**: cartella/USB, doppio clic, niente amministratore). Creano le tabelle **cliccando**, poi scrivono SQL. È ciò che si trova nei pannelli di hosting veri.
- **Piano B in cloud:** **Neon** o **Supabase** (un database Postgres online, senza installare niente — account del docente, sono minorenni).

Aggancio: è il database dei **prodotti dello shop** → si tocca il database *dentro* un progetto vero, non come esercizio a vuoto.

2. Costruire uno shop e-commerce funzionante (demo)

Cosa devono fare: realizzare un piccolo **negozio online funzionante** con prodotti e foto scelti da loro, e ottenerne un **link da mostrare**.

Perché è irrinunciabile: è “*la parte che tocca*” — li aggancia perché è roba loro, si vede e si mostra. Ed è il progetto che **unisce più competenze**: database (i prodotti) + pagina web + grafica.

Nota pratica: con i **minorenni non si attivano pagamenti reali** (serve un conto aziendale di un adulto) → si fa uno shop **demo funzionante**, tutto tranne l’incasso vero.

Strade (da dosare sulla classe):

- **Senza codice, risultato subito:** Big Cartel / Square Online / Store.link.
- **Con il codice (via software):** un mini-shop in HTML/CSS + un po’ di logica, da “remixare” e pubblicare su Glitch o Replit.

*È il punto da cui **ripartiamo la prossima sessione**.*

Storia delle versioni (per noi)

Versione	Data	Cosa è cambiato
0.1	27/07/2026	Nasce l'elenco delle cose irrinunciabili. Primi due punti: (1) toccare un database vero e scrivere SQL (sqliteonline → phpMyAdmin/MariaDB via XAMPP portable → Neon/Supabase come piano B), legato allo shop; (2) costruire uno shop e-commerce funzionante (demo).

Il Mio Negozio Online — Guida per i ragazzi

Versione 1.4 · 16/08/2026 · Parte: Classe 1 — Informatica

Un negozio vero, con database ed email — costruito da te

Alla fine di questo progetto avrai un **negozio online** con un **link tuo** da aprire sul telefono e mostrare a casa. Un negozio vero: i prodotti stanno in un **database in cloud** (il magazzino della classe), e quando qualcuno “compra” ti arriva un’**email con l’ordine**. Tutto **gratis** e **senza installare niente**.

Come è fatto (tre pezzi che lavorano insieme):

- **La vetrina** = la pagina che si vede (i prodotti, il carrello).
- **Il database** = il magazzino dove sono scritti i prodotti.
- **L’email** = l’avviso che ti arriva quando qualcuno ordina.

Com'è fatto il tuo negozio

tre pezzi che lavorano insieme



Schema del negozio: il cliente apre la vetrina (il sito), il database le manda i prodotti e la vetrina invia l'ordine per email.

Facciamo tutto a **piccole tappe**: a ogni tappa qualcosa **funziona** e lo puoi **mostrare**. Se ti blocchi, nessun problema: sbagliare è normale, si torna indietro con un clic.

Ti serve solo: un computer con un browser (Chrome/Edge) e un **account GitHub** (il prof ti dice come averlo). Il **database** lo ha già preparato il prof per tutta la classe: ti darà due valori da incollare. Niente da installare.

Ti serve anche il file di partenza: `modello-negozio.html`. Lo trovi nel repository del corso (cartella `classe-1/negozio-online/`) oppure te lo dà il prof. Scaricalo sul computer prima di cominciare.

TAPPA 1 — Metti il negozio ONLINE (la prima vittoria) 🌐

Obiettivo: avere un **link** con il tuo negozio che funziona (con dei prodotti di esempio). Ci arriviamo in pochi minuti.

1A • Crea lo spazio del negozio su GitHub

1. **[BROWSER]** vai su **github.com** e accedi al tuo account.
2. In **alto a destra**, clicca il **+** → poi **New repository**.
3. Alla voce **Repository name**, scrivi un nome, per esempio:

mio-negozio

1. Poco sotto, lascia selezionato **Public** (serve per avere il link gratis).
2. In fondo, clicca il bottone verde **Create repository**.

The screenshot shows the GitHub 'Create a new repository' page. The browser address bar shows 'github.com/new'. The page title is 'Create a new repository'. Below the title, it says 'Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository.](#)' and 'Required fields are marked with an asterisk (*)'.

The 'General' section (step 1) has two main fields: 'Owner *' with a dropdown menu showing 'nicolaregge-pulse' and 'Repository name *' with a text input field containing 'mio-negozio'. Below the 'Repository name' field, there is a red error message: 'mio-negozio already exists in this account'. Below this, there is a suggestion: 'Great repository names are short and memorable. How about [curly-memory](#)?'. There is also a 'Description' text area with a character count '0 / 350 characters'.

The 'Configuration' section (step 2) has a 'Choose visibility *' dropdown menu with 'Public' selected. Below it, there is a text area for 'Add README'.

La pagina "New repository" con il nome del negozio scritto e l'opzione "Public" selezionata.

1B • Carica il file del negozio

1. Nella pagina appena aperta, nel riquadro azzurro in basso, clicca il link blu **uploading an existing file**.
2. **Trascina** dentro l'area grande il file **modello-negozio.html**.
3. **Importante:** GitHub vuole che il file si chiami **index.html**. In alto, sopra il file caricato, c'è una casellina con il nome: cancella **modello-negozio.html** e scrivi:

index.html

1. Scendi in fondo e clicca il bottone verde **Commit changes**.

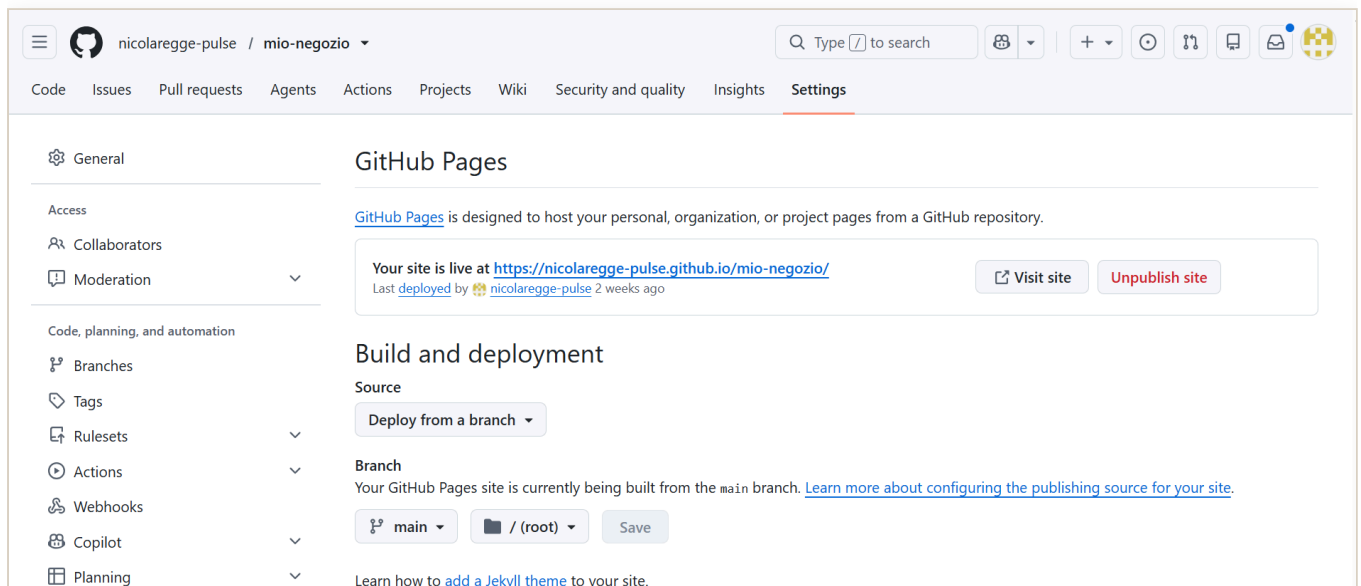
 **Qui va uno screenshot**

La casella con il nome del file cambiato in "index.html", prima di fare Commit.

salva la foto come: **negozio-02-rinomina-index.png**

1C • Accendi il link (GitHub Pages)

1. In alto nella pagina del repository, clicca **Settings** (l'ingranaggio).
2. Nel menu a **sinistra**, clicca **Pages**.
3. Alla voce **Source**, scegli **Deploy from a branch**.
4. Sotto, alla voce **Branch**, apri il menu e scegli **main**.
5. Lascia la cartella su **/ (root)** e clicca **Save**.



La pagina "Pages" con Source "Deploy from a branch", il ramo "main" e la cartella "/" (root)".

1D • Apri il tuo negozio

1. Aspetta **un minuto**, poi **ricarica** la pagina (tasto **F5**).
2. In alto compare un riquadro con “Your site is live at...” e un indirizzo tipo `https://iltuonome.github.io/mio-negozio/`.
3. **Clicca quell’indirizzo**: si apre il tuo negozio.

 **Il mio negozio** Carrello: 0 · Totale: € 0.00

 Database non ancora collegato: stai vedendo prodotti di esempio. Metti i due valori (indirizzo e chiave) per i prodotti veri.



Maglietta
€ 12.90

Aggiungi al carrello



Scarpe
€ 39.90

Aggiungi al carrello



Cappellino
€ 9.90

Aggiungi al carrello



Zaino
€ 24.90

Aggiungi al carrello

La tua cassa

Totale: € 0.00

Concludi l'ordine

Il negozio aperto nel browser, con i prodotti di esempio e il carrello.

 **FATTO!** Il tuo negozio è **online**. Aprilo sul telefono e fallo vedere a un compagno. Prova ad aggiungere prodotti al carrello: il totale si aggiorna. (I prodotti sono ancora di esempio: nella Tappa 2 arrivano quelli veri.)

TAPPA 2 — Collega il database della classe

Obiettivo: far arrivare nel tuo negozio i **prodotti veri**, presi dal database.

*Cos'è il database? È il magazzino dove sono scritti i prodotti (nome, prezzo, immagine). Il prof ne ha preparato **uno per tutta la classe** e ve lo mostra dal vivo. A te basta **collegarti**: non devi crearlo tu.*

*La chiave che ti dà il prof è **di sola lettura**: puoi **vedere** i prodotti, ma non puoi rovinarli. Tranquillo, non rompi niente.*

2A • Fatti dare i due valori dal prof

1. Chiedi al prof i **due valori** del database della classe:
2. l'**indirizzo** (comincia con `https://` e finisce con `.supabase.co`);
3. la **chiave pubblica** (comincia con `sb_publishable_...`).

2B • Mettili nel file del negozio

1. `[BROWSER — GitHub]` vai nel tuo repository `mio-negozio` → scheda `Code` → clicca il file `index.html`.
2. In alto a destra sopra il codice, clicca l'iconcina della **matita**  (“Edit this file”).
3. Cerca queste due righe (verso la metà del file):

```
const SUPABASE_URL = ""; // <-- CAMBIA QUI
const SUPABASE_KEY = ""; // <-- CAMBIA QUI
```

1. **Incolla** i due valori del prof **tra le virgolette**, così:

```
const SUPABASE_URL = "https://xxxxx.supabase.co";
const SUPABASE_KEY = "sb_publishable_xxxxx";
```

1. In alto a destra, clicca il bottone verde `Commit changes...` → poi di nuovo `Commit changes`.

```

69 // -----
70 const SUPABASE_URL = "https://dvphtapdixfovvdqnc.k.supabase.co"; // <-- Project URL
71 const SUPABASE_KEY = "sb_publishable_UeA0oWTjizAoQkBELxX9Q_e6Vznsvd"; // <-- chiave pubblica (publishable)
72
73 // Email dove ricevere gli ordini (ci pensa il servizio gratuito FormSubmit)

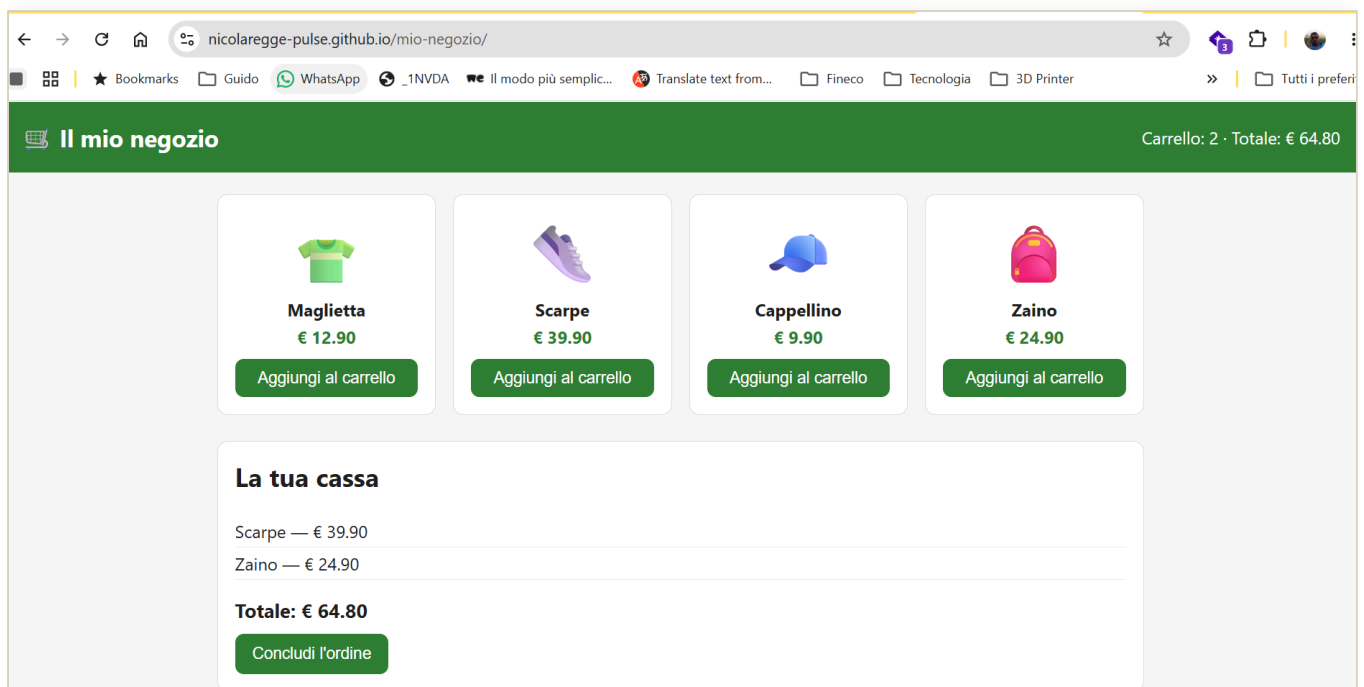
```

Le due righe SUPABASE_URL e SUPABASE_KEY con i valori del prof incollati tra le virgolette.

2C • Guarda il risultato

1. Aspetta un minuto, apri il tuo negozio e **ricarica** (**Ctrl + F5**).

✓ **FATTO!** Ora i prodotti arrivano dal **database della classe**. Prova “wow”: quando il prof cambia un prodotto nel database, ricaricate i vostri negozi... e cambia in **tutti** insieme! Ecco cos’è un database condiviso.



Il negozio con i prodotti VERI arrivati dal database della classe.

TAPPA 3 — Ricevi gli ordini via email



Obiettivo: quando qualcuno preme “*Concludi l'ordine*”, ti arriva un’**email**. Usiamo un aiutante gratuito che si chiama **FormSubmit**.

Come viaggia un ordine



Come viaggia un ordine: premi "Concludi l'ordine", passa da FormSubmit e arriva come email a te con nome, prodotti e totale.

3A • Metti la tua email nel file

1. [BROWSER — GitHub] nel tuo repository, apri `index.html` e clicca la **matita** .
2. Cerca questa riga:

```
const EMAIL_ORDINI = ""; // <-- CAMBIA QUI
```

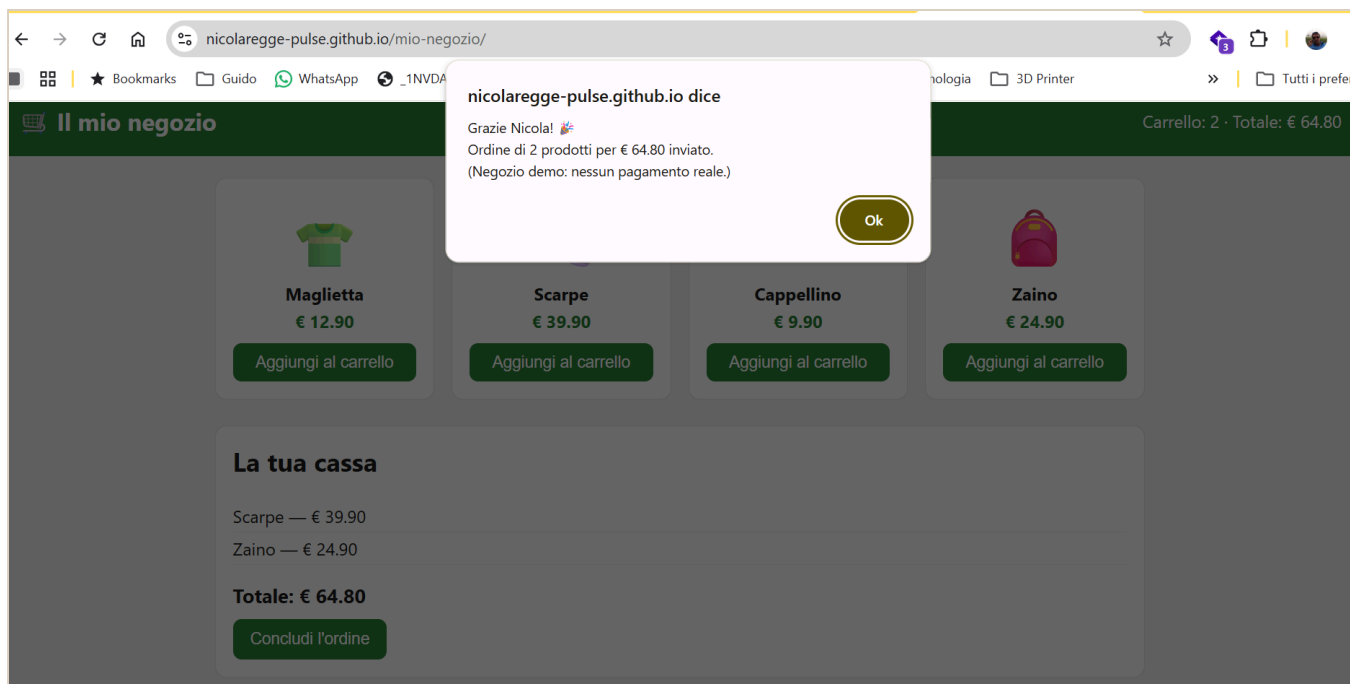
1. Scrivi la tua email **tra le virgolette**, così:

```
const EMAIL_ORDINI = "iltuonome@esempio.it";
```

1. Clicca `Commit changes...` → `Commit changes`.

3B • Prova l'ordine

1. Aspetta un minuto, apri il negozio e **ricarica**.
2. Aggiungi qualche prodotto al carrello e premi `Concludi l'ordine`.
3. Scrivi il tuo **nome** quando te lo chiede e conferma.



Il messaggio "Grazie, ordine inviato" che compare subito dopo aver concluso l'ordine.

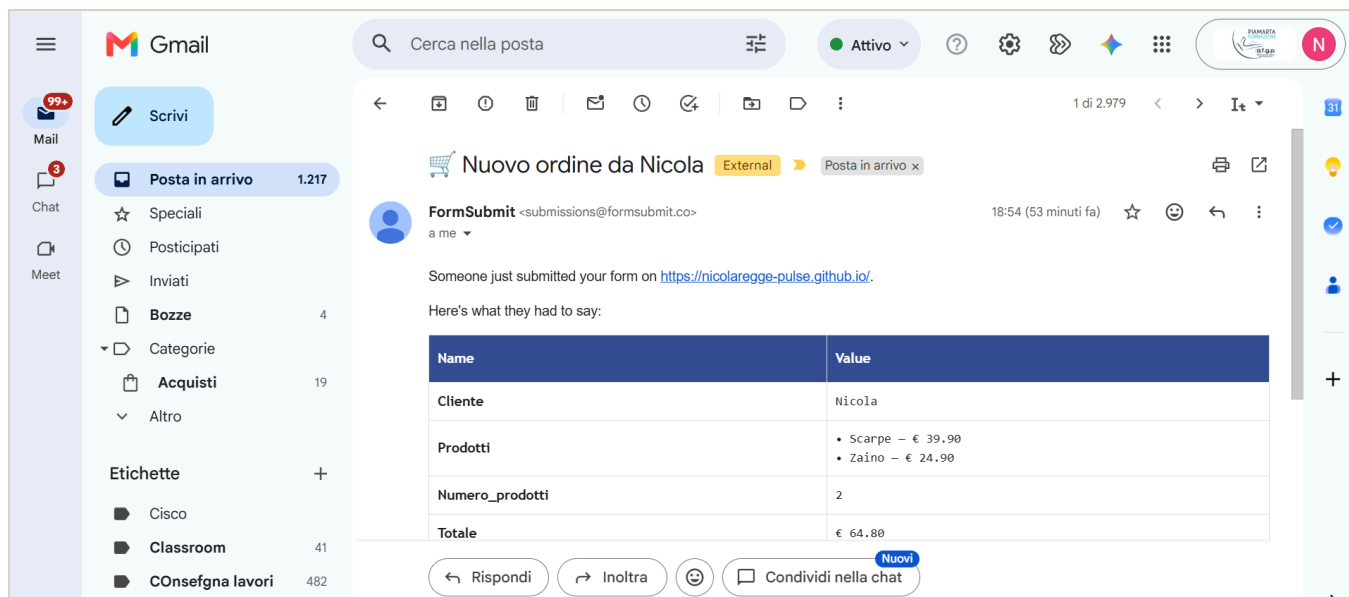
1. **La prima volta** ti arriva un'email da **FormSubmit** con un bottone tipo **Activate**: aprila (controlla anche lo **spam**) e cliccalo.

 **Qui va uno screenshot**

L'email di FormSubmit con il bottone "Activate" da cliccare la prima volta.

salva la foto come: `negozio-07-attiva-email.png`

1. Fai un **secondo** ordine di prova: adesso ti arriva l'**email con il riepilogo** (nome, prodotti, totale). 🎉



L'email con il riepilogo dell'ordine: nome del cliente, prodotti e totale.

✓ **FATTO!** Il tuo negozio è **completo**: prodotti dal database + ordini via email. Roba da tecnico vero.

⚠ **Nota:** è un negozio **demo**, non incassa soldi veri (per farlo serve un conto aziendale di un adulto). Va benissimo così per imparare e mostrare.

TAPPA 4 — Fallo tuo

Adesso rendilo **tuo davvero**:

- **Il nome:** nel file `index.html`, cambia la scritta dentro `<h1> Il mio negozio</h1>` (matita  → cambia → commit).
- **Il colore:** cerca `--colore: #2e7d32;` e cambia il codice colore (es. `#c0392b` rosso, `#8e44ad` viola).
- **I prodotti:** sono nel database della classe, uguali per tutti. Se vuoi dei prodotti **solo tuoi**, chiedi al prof: si può fare in un secondo momento.

✓ **Mostralo!** Fai uno screenshot del tuo negozio e mettilo nel tuo **quaderno**. Scrivi due righe: cos'è, come funziona, cosa hai cambiato tu.



Il negozio personalizzato: nome e colore scelti dal ragazzo.

La prova del nove

Sai **spiegare a voce**, con parole tue:

- dove stanno i **prodotti** (nel database della classe) e come fanno ad arrivare in vetrina?
- cosa succede quando premi “**Concludi l’ordine**”?

Se sai raccontarlo, **hai capito davvero** — ed è quello che conta.

Se qualcosa non va (succede a tutti)

- **Il link non si apre / pagina bianca:** aspetta un altro minuto, poi ricarica con `Ctrl + F5`. Controlla che il file si chiami **esattamente** `index.html`.
- **I prodotti sono ancora quelli di esempio:** controlla di aver incollato i due valori del prof **tra le virgolette** e di aver fatto **Commit**. Aspetta un minuto e ricarica.
- **L'email non arriva:** controlla lo **spam**; ricorda l'**attivazione** (la prima email di FormSubmit); controlla che la tua email nel file sia scritta giusta.

*Nessun errore ti fa danno: il tuo lavoro è salvato a ogni passo. Un bug è normale — **capita a tutti i programmatori, anche ai più bravi.***

Il Mio Negozio Online — Piano-lezione

Versione 1.0 · 09/08/2026 · Parte: Classe 1 — Informatica

In breve

I ragazzi costruiscono un **negozio online vero** (con un link da mostrare a casa), collegato a un **database della classe** e con gli **ordini via email**. Si fa in **circa 3 lezioni da un'ora**, e a ogni lezione ognuno porta a casa una **vittoria mostrabile**. Nessuno resta fuori: chi va piano si ferma alla prima tappa (negozio online) ed è già una vittoria; chi vola personalizza e aiuta i compagni.

Il filo è sempre lo stesso: **Vinci subito · Fallo tuo · Mostralo.**

Prima di iniziare — cosa prepara il prof (una volta sola)

1. **Il database condiviso su Supabase** (già fatto): tabella `prodotti` con la policy di **sola lettura**.
Consiglio: metti prodotti simpatici, magari a tema classe, così è più loro.
2. **I due valori da consegnare** ai ragazzi (li trovi nelle note del docente, `README.md`): l'**indirizzo** (`https://...supabase.co`) e la **chiave pubblica** (`sb_publishable_...`, di sola lettura). Scrivili alla lavagna o in un messaggio in Classroom.
3. **Il file di partenza** `modello-negozio.html`: mettilo dove i ragazzi lo prendono facile (link al repository, Drive o Classroom).
4. **Gli account GitHub**: verifica che ogni ragazzo ne abbia uno (o falli creare nella lezione zero).
5. **Un negozio “esempio” tuo già online**: da mostrare all’inizio come traguardo (“ecco dove arriviamo”).

La scaletta (3 lezioni)

Lezione 1 — “Il mio negozio è ONLINE” (Tappa 1)

Obiettivo: ognuno ha un **link** che funziona (con prodotti di esempio).

Tempo	Cosa
5'	Mostri il tuo negozio-esempio: “oggi ognuno fa il suo, con un link vero”.
30'	Insieme: crea repository → carica <code>index.html</code> → accendi GitHub Pages.
15'	Ognuno apre il suo link e lo mostra al compagno di banco.
10'	Chiusura: “FATTO — il tuo negozio è online”. Screenshot nel quaderno.

Vittoria: un link vero da mostrare. Nessuno esce senza il suo negozio online.

Lezione 2 — “I prodotti veri: il database” (Tappa 2)

Obiettivo: nel negozio compaiono i **prodotti veri**, dal database della classe.

Tempo	Cosa
15'	Tu spieghi il database dal vivo (canovaccio qui sotto).
25'	Distribuisci i due valori ; i ragazzi li incollano nel loro file.
10'	Il “ wow ”: cambi un prodotto nel database → tutti ricaricano → cambia in tutti i negozi.
10'	Chiusura + quaderno.

Lezione 3 — “Ordini via email + fallo tuo” (Tappe 3-4)

Obiettivo: gli ordini arrivano per **email**, e ognuno **personalizza** il suo.

Tempo	Cosa
20'	Ognuno mette la sua email , prova un ordine, attiva FormSubmit, riprova.
20'	Fallo tuo: cambia nome e colori del negozio.
20'	Mostralo + prova del nove: racconta a voce cosa fa il suo negozio; screenshot nel quaderno.

Canovaccio — spiegare il database dal vivo (10-15 min)

Al proiettore, sul **tu** Supabase. Poche cose, concrete:

1. **“Dove stanno i prodotti?”** → apri **Table Editor** → tabella `prodotti`. Fai vedere: ogni **riga** è un prodotto, ogni **colonna** un’informazione (nome, prezzo, foto). *Analogia*: è un **magazzino** (o il registro di classe: righe = alunni, colonne = dati).
2. **“Come glielo chiediamo?”** → apri **SQL Editor** e scrivi dal vivo: `sql select * from prodotti;`
Premi **Run**: tornano tutti. Poi: `sql select * from prodotti where prezzo < 20;` Tornano solo alcuni. Messaggio: **“SQL = fare domande al magazzino.”**
3. **“Chi può cambiarli?”** → spiega la **sola lettura**: i ragazzi **guardano**, solo il prof **modifica**. Così nessuno rovina il lavoro degli altri.
4. **Il “wow” (fallo alla Lezione 2)**: cambia un **prezzo** nel Table Editor → fai ricaricare un negozio a caso → è cambiato. *“Un database è vivo: cambio qui, cambia dappertutto.”*

*Se qualcuno chiede “perché non nel file?”: perché così i prodotti stanno in **un posto solo** e li aggiorni una volta per tutti. È il senso del database.*

Se lavori a gruppi (opzionale, 2-4 ragazzi)

Ruoli **a rotazione**, così tutti provano tutto:

- **Vetrina:** crea il repository e pubblica su GitHub Pages.
- **Database:** incolla i due valori e verifica i prodotti.
- **Grafica:** sceglie nome e colori.
- **Ordini:** mette l'email e prova la cassa.

Poi si cambia, così nessuno si nasconde dietro i più bravi e ognuno tocca ogni pezzo. (Si lega bene a **Git**: ognuno lavora sul suo pezzo, poi si uniscono.)

Gestire i ritmi diversi (importante per questa classe)

- **Chi va piano:** basta arrivare alla **Tappa 1** (negozi online con prodotti di esempio). È già una vittoria vera e mostrabile. **Nessuno resta fuori.**
- **Chi vola:** personalizza colori e testo del bottone, aggiunge prodotti (con te), oppure fa da **tutor** a un compagno (spiegare consolida).
- **Errore = zero vergogna:** si annulla con un clic, il bug è normale — “*capita a tutti i programmatori, anche ai più bravi*”.

Valutazione (coerente col corso)

1. **Il negozio che funziona è il biglietto d'ingresso, non il voto.**
2. **Il voto nasce dalla prova dal vivo:** lo **spiega a voce** (cosa fa, dove stanno i prodotti, cosa succede all'ordine), **oppure** gli dai il suo file con **un piccolo errore** e lo rimette a posto lì per lì.
3. **Prova del nove:** se lo sa **raccontare con parole sue**, la competenza c'è.

Checklist da tenere in aula

- [] PC con browser + proiettore
- [] I **due valori** del database (indirizzo + chiave pubblica)
- [] Il file `modello-negozio.html` raggiungibile dai ragazzi
- [] Gli **account GitHub** pronti
- [] Il tuo **negozio-esempio** già online da mostrare

Il Manuale di Godot

Versione 0.5 · 26/07/2026 · Parte: Corso Godot / GDScript

Come si valutano i compiti

Le regole del gioco, chiare fin da subito.

Qui non ti freghiamo: sai **in anticipo** come funziona la valutazione. Leggila una volta, poi lavora tranquillo.

1. Il codice che funziona è il biglietto d'ingresso, non il voto. Consegnare il gioco che gira ti fa “entrare alla prova”. Ma il codice **da solo** non fa il voto: puoi copiarlo o fartelo scrivere... e allora non direbbe niente su di te.

2. Il voto nasce dalla prova dal vivo, da solo. Uno di questi due, o tutti e due:

- **Me lo spieghi:** racconti **a parole tue** cosa fa il tuo codice.
- **Il gioco rotto:** ti do il tuo gioco con **2-3 errori nascosti** dentro, e tu lo **rimetti in piedi lì per lì** — da solo, senza AI e senza chiedere ai compagni.

Se hai capito, li fai in pochi minuti. Se hai solo incollato, ti blocchi. Ecco perché **capire conviene**.

3. Il patto con l'AI e con i compagni. Puoi usarli per **imparare**: capire un errore, farti spiegare, avere uno spunto. Non per **consegnare senza capire**. La prova del nove è sempre la stessa: **lo sai spiegare e riparare da solo?**

4. Zero vergogna. Usare gli aiuti, come i 4 livelli, l'AI o un compagno, è **permesso e normale** — non è imbrogliare. Anche sbagliare è normale: **capita a tutti i programmatori**, pure ai più bravi. L'unica cosa che conta è che, alla fine, **tu abbia capito**.

Scrivere in Markdown

La tua dispensa, fatta con due segnetti.

Markdown, si dice “*marc-daun*”, è un modo per scrivere **testo normale** e aggiungere qualche **segnetto** che dice *come deve apparire*: questo è un titolo, questa parola è in grassetto, questo è un elenco.

***Il ponte con quello che conosci:** in Word premi il bottone grassetto; qui il bottone non c'è, il grassetto lo **scrivi tu** mettendo due asterischi attorno alla parola. Tutto qui. I segnetti li scrivi tu, ma nel risultato finale **non si vedono**.*

La tabella dei segnetti

Vuoi ottenere...	Scrivi così
Un titolo grande	# Il mio titolo
Un titolo più piccolo	## Sottotitolo — più #, più piccolo
Grassetto	**parola** — due asterischi prima e dopo
<i>Corsivo</i>	*parola* — un asterisco prima e dopo
Un punto elenco	- prima cosa — trattino più spazio
Un elenco numerato	1. prima cosa
Un nome tecnico / codice in riga	`_process` — un apice basso prima e dopo
Un'immagine, un tuo screenshot	![il mio gioco](immagini/es1.png)
Una nota/riquadro	> Attenzione: ricordati di salvare!

L'apice basso ```, in inglese *backtick*, su tastiera italiana si fa con **Alt Gr** + `'`, il tasto dell'apostrofo vicino allo `0`.

Un blocco di codice

Metti **tre apici bassi** prima e **tre** dopo: così il codice viene mostrato bello incolonnato.

```
```gdscript
func _ready():
 print("Ciao!")
```
```

Le 4 regole d'oro

1. **Riga vuota tra un paragrafo e l'altro.** Se non la metti, le frasi si attaccano tutte insieme.
2. **Uno spazio dopo # e dopo -.** `#Titolo` non funziona, `# Titolo` sì.
3. **I segnetti non si vedranno:** servono solo a dare la forma. Non spaventarti se nel testo grezzo sembrano strani.
4. **Bloccato? Fatti aiutare bene dall'AI.** Scrivi con **parole tue**, poi chiedi “*mettimi questo in Markdown*”, e **guarda come l'ha fatto** — così impari il segnetto per la prossima volta. Ricorda il patto: l'AI ti aiuta a *formattare*, il pensiero resta tuo.

Come parti da un modello

Non parti mai da un foglio bianco: c'è un **modello** già pronto, in inglese *template*, con i titoli e i posti da riempire.

Cos'è il “repository”? È solo una parola tecnica per dire **la cartella del corso su GitHub**, dove sono raccolti tutti i file: il manuale, gli esercizi, i modelli. È lo stesso posto che gestiamo con **Git**. Da browser lo apri come un sito qualsiasi: cartelle e file su cui puoi cliccare.

Ecco come apri il modello e te ne fai **una copia tua**:

1. `[BROWSER]` apri la pagina del corso su GitHub. L'indirizzo esatto te lo do io.
2. Nella lista, clicca prima la cartella `manuale`, poi il file `quaderno-studente-TEMPLATE.md`: si apre e vedi il testo del modello.
3. In alto a destra, sopra il testo, clicca l'iconcina `Copy raw file`, che copia tutto il contenuto in un colpo solo.
4. Torna nel **TUO** repository, la tua copia personale → bottone `Add file` → `Create new file`.
5. Dai un nome che finisce con `.md`, per esempio `es1.md`.
6. **Incolla** con `Ctrl + V` il modello, poi **riempi i vuoti** con le tue cose.
7. Bottone verde `Commit changes` per salvare. Fatto!

Come metto un mio screenshot

Uno screenshot è una **foto dello schermo**. Attenzione: appena lo fai, finisce solo nella memoria temporanea, i cosiddetti “appunti” — **non è ancora un file** sul computer. Ecco tutti i passaggi:

1. [Windows] premi **Win + Shift + S**: lo schermo si scurisce, **trascina** un riquadro attorno al tuo gioco. L'immagine viene copiata.
2. In basso a destra compare un **avviso: cliccaci sopra** → si apre lo **Strumento di cattura**.
3. Lì clicca l'iconcina del **dischetto** per salvare, scegli una cartella che **ricordi bene**, per esempio il **Desktop**, e un nome che finisce con **.png**, per esempio **es1.png**. Ora è un **file** sul tuo computer.
4. [BROWSER] nel tuo repository apri la cartella **immagini/** → bottone **Add file** → **Upload files** → **trascina** dentro il tuo **es1.png**.
5. La riga nel modello è **già pronta**: **![il mio gioco](immagini/es1.png)** → l'immagine comparirà da sola.

Prova del nove: se guardi la tua pagina e vedi il titolo grande, il grassetto e il tuo screenshot al posto giusto... **ce l'hai fatta!**

Cos'è Godot, il parente di Lazarus

Godot è un programma gratuito per creare **giochi** e app interattive. È molto simile, come spirito, a **Lazarus**: entrambi sono ambienti gratuiti dove **componi qualcosa a schermo e ci attacchi del codice**.

La “stele di Rosetta” Lazarus → Godot:

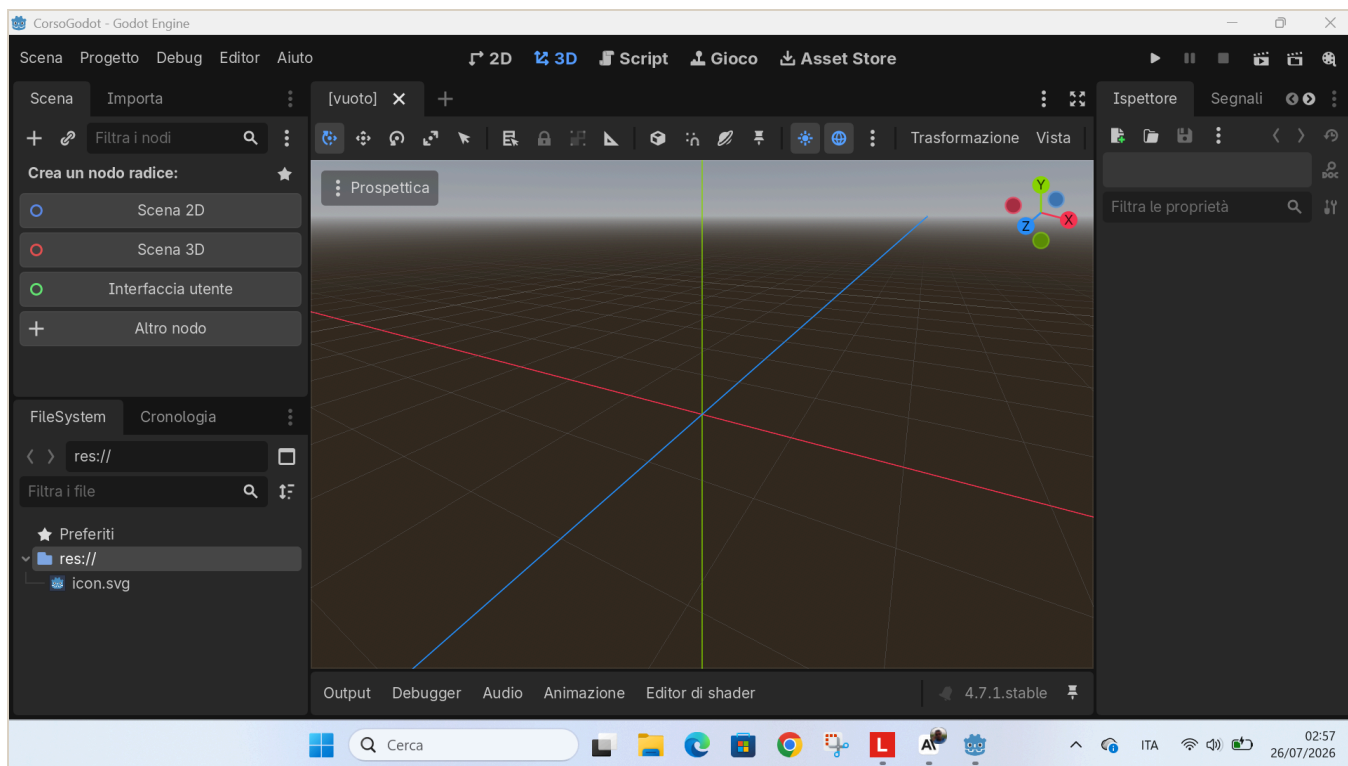
| In Lazarus, che già conosci | In Godot, la novità |
|--|---|
| Progetto | Progetto , identico |
| Form , la finestra | Scena , un <i>albero di nodi</i> più potente |
| Componenti : TButton, TEdit... | Nodi : Button, LineEdit, Label... |
| Proprietà come Caption, nell'Object Inspector | Proprietà nell' Ispettore |
| Gestore evento , <code>Button1Click</code> | Segnale + funzione |
| Object Pascal , il linguaggio | GDScript , in stile Python |

La differenza più importante — il “game loop”: in Lazarus il programma è **fermo** finché non clicchi qualcosa. In Godot c'è una funzione, `_process(delta)`, che **gira da sola ~60 volte al secondo**, in continuazione. È questo che fa muovere le cose: personaggi, oggetti che cadono, animazioni.

*In una frase: **Lazarus reagisce, Godot pulsa.***

L'ambiente di sviluppo all'avvio

Quando apri un progetto nuovo, l'editor si presenta così: vista **3D** di default e scena ancora vuota.



L'editor di Godot appena aperto, con la scena vuota

*Per il nostro corso lavoreremo quasi sempre in **2D**: cliccheremo “**Scena 2D**” per iniziare. Lo vediamo nel Capitolo 1.*

I 4 concetti base di Godot

Se capisci questi quattro, capisci Godot:

| Concetto | Cos'è | Analogia LEGO |
|--|---|--------------------|
| Progetto | La cartella con dentro il file <code>project.godot</code> e tutto il gioco | La scatola del set |
| Nodo | Il mattoncino base: Sprite2D per un'immagine, Label per un testo, Timer per un cronometro | Un pezzo di LEGO |
| Scena | Tanti nodi messi ad albero, salvati in un file <code>.tscn</code> | Una costruzione |
| Script , il file <code>.gd</code> | Codice GDScript attaccato a un nodo, che gli dà comportamento | Le istruzioni |

Regola d'oro: in Godot **tutto è un nodo**; le scene sono nodi messi insieme; gli script danno vita ai nodi.

GDScript: il linguaggio

GDScript è la lingua che si parla **dentro** Godot. È stato fatto apposta per somigliare a **Python**: si legge facile, si usa il **rientro con TAB** per raggruppare le righe, **niente punto e virgola**.

Due funzioni speciali che Godot chiama da solo:

- `func _ready():` → eseguita **una volta**, all'avvio, per preparare le cose.
- `func _process(delta):` → eseguita **a ogni fotogramma**: è il game loop. `delta` = secondi passati dall'ultimo fotogramma; serve a muoversi in modo fluido su qualsiasi PC.

Esempio minimo, leggere le frecce e spostare qualcosa:

```
func _process(delta):  
    if Input.is_action_pressed("ui_left"):  
        posizione.x -= 200 * delta    # vai a sinistra  
    if Input.is_action_pressed("ui_right"):  
        posizione.x += 200 * delta    # vai a destra
```

Il nostro primo gioco: “Chirurgo Pasticcione”

Idea: un chirurgo maldestro fa cadere gli organi dal tavolo. Tu muovi il **vassoio** con le frecce ← → e li prendi al volo.

- organo preso → **+1 punto**
- organo per terra → **-1 vita**
- zero vite → **Operazione Fallita**, INVIO per riprovare

Cosa ci insegna:

- Creare nodi da codice: il vassoio, gli organi, le scritte.
- Il **movimento** con le frecce, usando input e `delta`.
- Le **collisioni**, quando il vassoio “tocca” un organo.
- Tenere lo **stato del gioco**, punti e vite, e mostrarlo a schermo.
- Un **Timer** che fa comparire gli organi a intervalli.

Il codice completo e commentato è in `godot/chirurgo-pasticcione/main.gd`.

Il percorso: dagli esercizi al “progetto boss”

Qui non si impara con la teoria astratta, ma **facendo**. Ogni esercizio insegna **un pezzo**; poi arriva un gioco più grande — il “**progetto boss**” — che mette insieme quei pezzi. Ecco la scala che stiamo salendo.

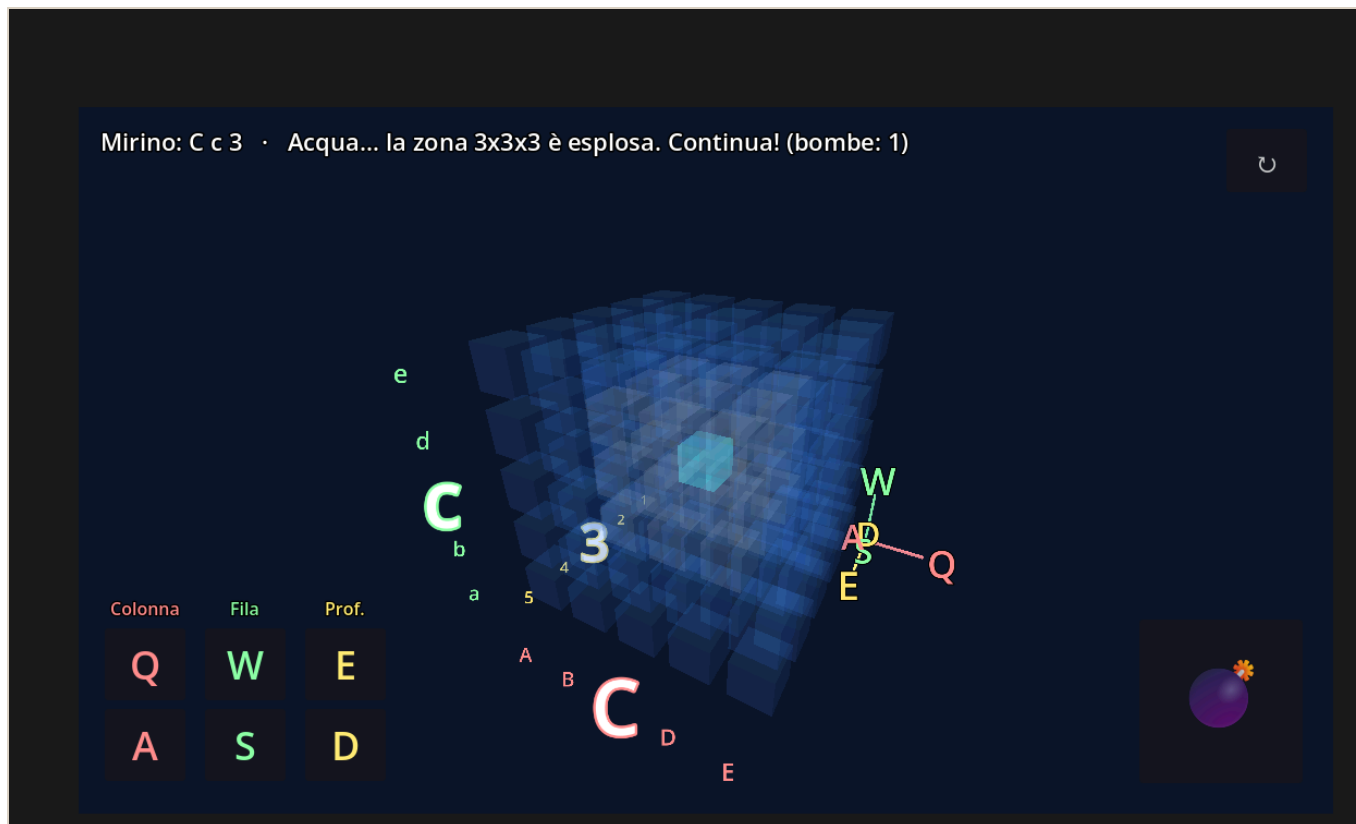
I gradini piccoli: gli esercizi dell’eserciziario

| Esercizio | Cosa impari | Il concetto sotto |
|---------------------------|-------------------------------|---|
| 1 • Il bottone che saluta | Un clic fa succedere qualcosa | Il segnale , il tuo <code>Button1Click</code> di Lazarus |
| 2 • Muovi il quadrato | Far muovere le cose da sole | Il game loop <code>_process(delta)</code> + input |
| 3 • Prendi la moneta | Un mini-gioco vero | Movimento + collisioni + punteggio |

Ognuno è **corto** e finisce con una **vittoria a schermo**: è fatto apposta così, per vincere subito e non mollare.

Cos’è un “progetto boss”

È un gioco **già pronto**, più grosso, che **non si copia riga per riga**. Si **apre, si gioca e si rende proprio**: cambi i colori, il titolo, ci metti una tua foto. È il **premio**: la cosa figa da mostrare subito. Il primo è “**Affonda la Bonomi**”: lo trovi nell’eserciziario e nella cartella `battaglia-navale-3d/`.



Il "progetto boss" Affonda la Bonomi: una battaglia navale in 3D, dentro un cubo d'acqua.

Dal 2D al 3D: cosa cambia nel boss

Finora abbiamo lavorato in **2D**: due coordinate, **x** orizzontale e **y** verticale, un `Vector2`. Il boss è in **3D**: si aggiunge una terza coordinate, **z**, la **profondità**, un `Vector3`. Il campo di gioco non è più una griglia piatta ma un **cubo** di celle.

Due idee nuove, ma **niente panico**:

- **Si costruisce tutto da codice**: le celle, le luci, la telecamera e i bottoni, invece che a mano nell'editor: sono sempre gli stessi **nodi**, solo tanti, creati con un ciclo `for`.
- Sono gli **stessi concetti di prima, in grande**: il **game loop** gira il cubo, l'**input** muove il mirino. Chi ha fatto gli esercizi 2 e 3 ha già visto tutto.

Come lo proponiamo ai ragazzi: la parte che riescono a fare

La regola d'oro: **ognuno deve poter salire almeno un gradino** e portarsi a casa una vittoria vera. Non si "finisce" il boss tutto in una volta: ci si torna più volte durante l'anno, un gradino per volta.

1. **Gioca** e scegli la difficoltà, con 4 è facilissimo. → *ci riescono tutti*.
2. **Cambia un colore** dell'acqua o del mirino: basta cambiare un numero nel codice. → facile, effetto immediato.

3. **Metti la tua foto**, o un meme, al posto di quella di default.

4. **Cambia il titolo** del gioco.

5. **Leggi una funzione piccola** e spiega **a voce** cosa fa, per esempio come si muove il mirino. → è la “prova del nove”.

6. **Per i più veloci**: cambia la potenza della bomba o aggiungi un secondo sottomarino.

Così **nessuno resta fuori**: chi è più indietro gioca e cambia un colore, ed è già una vittoria mostrabile; chi corre di più mette le mani nel codice. Vale sempre: **Vinci subito · Fallo tuo · Mostralo**.

Come useremo l'AI

L'AI è come la **calcolatrice in matematica**: aiuta, ma se non capisci cosa stai facendo non serve a niente.

- Usala per: capire un errore, farti spiegare un concetto, avere un suggerimento, uno **spunto da studiare e modificare**.
- Non usarla per: farti scrivere tutto e consegnarlo senza capirlo.
- **Prova del nove**: se sai **spiegare a voce, riga per riga**, il codice che presenti, la competenza c'è.

Changelog del manuale

| Versione | Data | Cosa è cambiato |
|----------|------------|--|
| 0.1 | 26/07/2026 | Prima stesura: Cap. 0 Godot vs Lazarus, Cap. 1 i quattro concetti, Cap. 2 GDScript e game loop, Cap. 3 Chirurgo Pasticcione, regola uso AI. |
| 0.2 | 26/07/2026 | Aggiunte due schede iniziali: Scheda 1 “Come si valutano i compiti” e Scheda 2 “Scrivere in Markdown”, con la tabella dei segnetti e come partire da un modello. |
| 0.3 | 26/07/2026 | Aggiunto il Capitolo 4 “Il percorso: dagli esercizi al progetto boss”: collega i 3 esercizi ai concetti, spiega cos’è un progetto boss e il passaggio 2D → 3D, e come proporre “Affonda la Bonomi” ai ragazzi a gradini, con screenshot. |
| 0.4 | 26/07/2026 | Stile: sottotitoli delle schede/capitoli resi come sottotitolo centrato più piccolo; blocchi di codice nero-su-bianco su fondo chiaro per stampare senza sprecare toner. |
| 0.5 | 26/07/2026 | Aspetto più sobrio e formale: rimosse tutte le icone/emoji; tolte le parentesi da titoli e scritte in grassetto; copertina senza emoji; istruzioni per principianti più complete (modello e screenshot); corretta una pagina vuota; le frasi tra virgolette non si spezzano più a fine riga. |

Eserciziario di Godot

Versione 0.5 · 26/07/2026 · Parte: Corso Godot / GDScript

Come funziona ogni esercizio

Ogni esercizio ha **4 livelli di aiuto**. Prova sempre da solo, e apri il livello successivo **solo se sei bloccato**:

1.  **Descrizione** — cosa devi ottenere.
2.  **Aiuto** — un indizio su come fare.
3.  **La scena** — quali nodi creare, i “mattoncini”.
4.  **Codice completo** — la soluzione da copiare/incollare.

*Regola: se copi il **Codice completo**, poi devi saper **spiegare a voce cosa fa, riga per riga**. Se lo sai spiegare, hai imparato lo stesso.*

Nel file `.md` i livelli 2–4 sono a scomparsa: clicca sul triangolino per aprirli. Nel PDF sono già aperti.

***Oltre agli esercizi numerati ci sono i “Progetti BOSS”:** giochi già pronti, più grossi, che non si copiano riga per riga — si **aprono, si giocano e si rendono propri**: cambi colori, titolo, ci metti una tua foto. Sono il premio: la cosa figa da mostrare subito agli amici. Il codice è già nel repository, lo capiremo un pezzo alla volta.*

***Nota per il docente:** per ora gli esercizi sono raccolti così come nascono, non in ordine di difficoltà. Più avanti li riordineremo per difficoltà o per quando si svolgono in classe. L’importante adesso è raccoglierli.*

Il bottone che saluta

Ponte da Lazarus: è il tuo `Button1Click` che cambia una `Caption`!

Descrizione

Crea una schermata con **un bottone** e **una scritta**. Quando premi il bottone, la scritta deve cambiare, per esempio da “...” a “Ciao! Mi hai premuto”.

Fallo tuo: scegli **tu** la frase del saluto e il **colore** della scritta — così il gioco è già tuo. Nessuno lo farà uguale al tuo!

Aiuto

- In Godot il bottone è il nodo **Button**, la scritta è il nodo **Label**.
- La proprietà `text` di Godot è come la **Caption** di Lazarus.
- L’evento “click” in Godot si chiama **segnale** `pressed`. Lo colleghi a una tua funzione con `bottone.pressed.connect(la_mia_funzione)`.

La scena — i nodi da creare

1. Nodo radice: **Node2D**, rinominalo `Main`.
2. Figlio: **Button** → rinominalo `BottoneCiao`.
3. Figlio: **Label** → rinominalo `Etichetta`.
4. Attacca uno **script** al nodo radice `Main`.

Codice completo

```
extends Node2D
```

```
# $NomeNodo = prende un nodo figlio per nome (come Button1 in Lazarus)
```

```
@onready var bottone: Button = $BottoneCiao
```

```
@onready var etichetta: Label = $Etichetta
```

```
func _ready() -> void:
```

```
    # Posizioniamo i due elementi così non si sovrappongono
```



```
bottone.position = Vector2(100, 100)
bottone.text = "Salutami!"           # <- come Button.Caption in Lazarus
etichetta.position = Vector2(100, 180)
etichetta.text = "..."
# FALLO TUO: scegli il colore della scritta, rosso verde blu da 0 a 1
etichetta.add_theme_color_override("font_color", Color(1, 0, 0)) # rosso
# Colleghiamo il "click" (segnale pressed) alla nostra funzione
bottone.pressed.connect(_quando_premo)

# Questa e' come il tuo Button1Click di Lazarus
func _quando_premo() -> void:
    # FALLO TUO: scrivi qui il TUO saluto
    etichetta.text = "Ciao! Mi hai premuto."
```

ESERCIZIO 2

Muovi il quadrato

Concetto nuovo: il game loop, cioè `_process`.

Descrizione

Fai comparire un **quadrato** che puoi muovere in tutte le direzioni con le **freccie** della tastiera.

Aiuto

- Un quadrato colorato semplice = nodo **ColorRect**.
- Il movimento va scritto in `_process(delta)`: e' la funzione che gira ~60 volte al secondo. In Lazarus non c'era: il programma stava fermo.
- Le frecce si leggono con `Input.is_action_pressed("ui_left")`, e allo stesso modo `ui_right`, `ui_up`, `ui_down`.
- Moltiplica sempre la velocita' per `delta`, cosi' va uguale su ogni PC.

La scena — i nodi da creare

1. Nodo radice: **Node2D**, rinominalo `Main`.
2. Figlio: **ColorRect** → rinominalo `Quadrato`.
3. Attacca uno **script** al nodo radice `Main`.

Codice completo

```
extends Node2D

@onready var quadrato: ColorRect = $Quadrato
const VELOCITA: float = 300.0 # pixel al secondo

func _ready() -> void:
    quadrato.size = Vector2(60, 60)
    quadrato.color = Color(0.3, 0.7, 1.0) # azzurro
    quadrato.position = Vector2(200, 200)
```

```
# _process gira a OGNI fotogramma: qui muoviamo il quadrato
func _process(delta: float) -> void:
    if Input.is_action_pressed("ui_left"):
        quadrato.position.x -= VELOCITA * delta
    if Input.is_action_pressed("ui_right"):
        quadrato.position.x += VELOCITA * delta
    if Input.is_action_pressed("ui_up"):
        quadrato.position.y -= VELOCITA * delta
    if Input.is_action_pressed("ui_down"):
        quadrato.position.y += VELOCITA * delta
```

ESERCIZIO 3

Prendi la moneta

Mette insieme: movimento + oggetto che cade + punteggio. Verso il gioco vero.

Descrizione

Un **cestino** in basso, che muovi con le frecce $\leftarrow \rightarrow$, e una **moneta** che cade dall'alto. Se la prendi col cestino fai **+1 punto** e la moneta riparte dall'alto in una colonna a caso. Mostra il punteggio a schermo.

Aiuto

- Cestino e moneta: due **ColorRect**. Il punteggio: un **Label**.
- Muovi il cestino in `_process`, come nell'Esercizio 2 ma solo sinistra/destra.
- Fai scendere la moneta ogni fotogramma: `moneta.position.y += velocita * delta`.
- Per capire se il cestino "tocca" la moneta usa i rettangoli: `Rect2(a.position, a.size).intersects(Rect2(b.position, b.size))`.
- Quando la moneta esce sotto o è presa, rimettila in alto a una `x` a caso.

La scena — i nodi da creare

1. Nodo radice: **Node2D**, rinominalo `Main`.
2. Figlio **ColorRect** → `Cestino`.
3. Figlio **ColorRect** → `Moneta`.
4. Figlio **Label** → `Punteggio`.
5. Attacca uno **script** al nodo radice `Main`.

Codice completo

```
extends Node2D
```

```
@onready var cestino: ColorRect = $Cestino
@onready var moneta: ColorRect = $Moneta
@onready var punteggio: Label = $Punteggio
```

```

const VELOCITA_CESTINO: float = 500.0
const VELOCITA_MONETA: float = 300.0

var punti: int = 0
var larghezza: float

func _ready() -> void:
    larghezza = get_viewport_rect().size.x
    # Cestino in basso
    cestino.size = Vector2(120, 24)
    cestino.color = Color(0.6, 0.4, 0.2)
    cestino.position = Vector2(larghezza / 2.0 - 60, get_viewport_rect().size.y -
60)
    # Moneta
    moneta.size = Vector2(30, 30)
    moneta.color = Color(1.0, 0.85, 0.1)
    _rimetti_in_alto()
    # Punteggio
    punteggio.position = Vector2(20, 20)
    _aggiorna_punteggio()

func _process(delta: float) -> void:
    # Muovi il cestino
    if Input.is_action_pressed("ui_left"):
        cestino.position.x -= VELOCITA_CESTINO * delta
    if Input.is_action_pressed("ui_right"):
        cestino.position.x += VELOCITA_CESTINO * delta
    cestino.position.x = clamp(cestino.position.x, 0, larghezza - cestino.size.x)

    # Fai scendere la moneta
    moneta.position.y += VELOCITA_MONETA * delta

    # Presa?
    if Rect2(cestino.position, cestino.size).intersects(Rect2(moneta.position,
moneta.size)):
        punti += 1
        _aggiorna_punteggio()
        _rimetti_in_alto()
    # Persa (uscita sotto)?
    elif moneta.position.y > get_viewport_rect().size.y:
        _rimetti_in_alto()

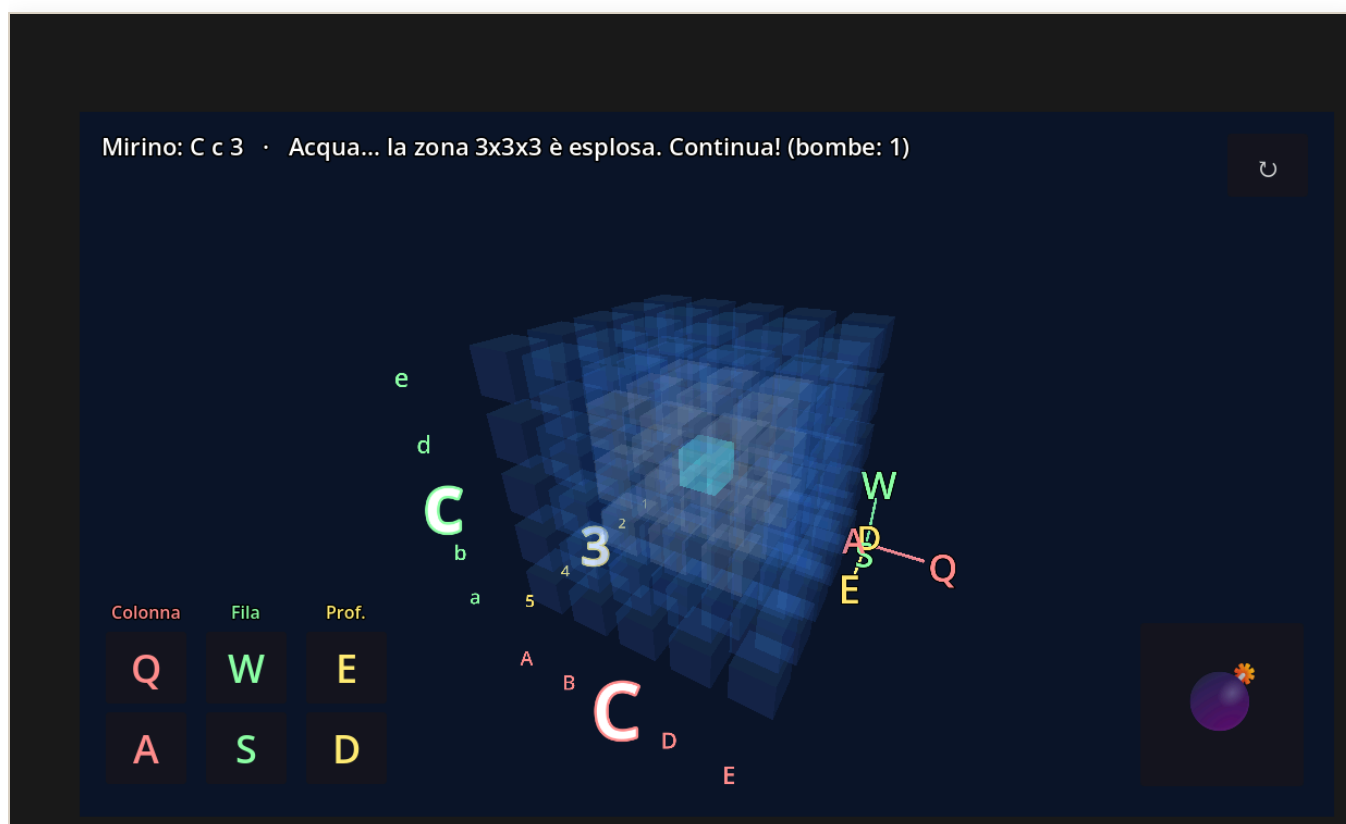
func _rimetti_in_alto() -> void:
    var x := randf_range(0, larghezza - moneta.size.x)
    moneta.position = Vector2(x, -moneta.size.y)

func _aggiorna_punteggio() -> void:
    punteggio.text = "Monete: %d" % punti

```

Affonda la Bonomi

Il primo “progetto boss”: una battaglia navale in 3D già giocabile. Si apre, si gioca, si rende proprio.



Affonda la Bonomi in azione: il cubo d'acqua con il mirino verde, le coordinate scritte attorno al cubo e, in basso a sinistra, i comandi colorati dei tre assi (Colonna Q/A, Fila W/S, Profondità E/D).

Descrizione

La solita battaglia navale, ma **in tre dimensioni**: al posto della griglia a righe e colonne c'è un **cubo** di celle d'acqua. Dentro è nascosto **un sottomarino**. Muovi un **mirino** e lanci una **bomba di profondità**: esplode e colpisce una zona **3×3×3** attorno al punto. Se il sottomarino è lì → **COLPITO!** Questo gioco è **già fatto**: il tuo compito è **aprirlo, giocarci e farlo tuo**.

● Aiuto — come aprirlo e giocarci

1. [APP – Godot] finestra iniziale, il Gestore progetti, in alto a destra **Importa** → scegli la cartella **battaglia-navale-3d** e il file **project.godot** → **Importa e modifica**.

2. Premi **F5** per eseguire. All'avvio scegli **quanti cubi per lato**, da **4** facilissimo, fino a **10** difficile.
3. Comandi, con le lettere disposte come **tre colonne della tastiera**:
4. **Q / A** = Colonna, in rosso · **W / S** = Fila, in verde · **E / D** = Profondità, in giallo
5. **SHIFT + le stesse lettere** = gira il cubo · **dito/mouse trascinato** = gira il cubo
6. **SPAZIO** = lancia la bomba · **↵ / INVIO** = rigioca e richiede di nuovo la difficoltà

● Fallo tuo — la parte più importante

Apri `battaglia-navale-3d/main.gd` e cambia queste cose per rendere il gioco **tuo**. Dopo ogni modifica premi **F5** e guarda l'effetto:

- **I colori dell'acqua e del mirino:** in alto trovi righe tipo `const COL_ACQUA := Color(...)` e `const COL_MIRINO := Color(...)`. Cambia i tre numeri, rosso verde blu da 0 a 1, e avrai il **tuo stile**.
- **La tua foto al posto di Serena:** metti un file `serena.jpg`, (una tua foto o un meme) nella cartella `battaglia-navale-3d/`: comparirà quando affondi il sottomarino, lampeggiando con il teschio dei pirati.
- **Il titolo del gioco:** in `project.godot`, alla voce `config/name="..."`, scrivi il **nome che vuoi tu**.
- **La difficoltà di partenza / la potenza della bomba:** prova a cambiare `RAGGIO_BOMBA`: 1 è la zona 3×3×3, 2 è la zona 5×5×5, molto più potente.

Mostralo: quando l'hai personalizzato, fai una partita davanti a un compagno. “Questo l'ho fatto **io**” vale più di qualsiasi voto.

● Il codice completo — dov'è e com'è fatto

Il codice **c'è già tutto** ed è versionato nel repository, nel file `battaglia-navale-3d/main.gd`, circa 600 righe. Non va copiato a mano: è il nostro **progetto boss**, lo leggeremo **un pezzo alla volta**.

Le idee sono le **stesse degli esercizi precedenti**, portate in 3D:

- il **game loop** `_process(delta)` per girare il cubo, come nell'Esercizio 2;
- **leggere i tasti** con `Input.is_key_pressed(...)`, come nel muovere il quadrato;
- **costruire tutto da codice:** celle, luci, telecamera, bottoni, invece che a mano.

*Regola d'oro, valida anche qui: se sai **spiegare a voce** cosa fa un pezzo di codice, quel pezzo è tuo. Partiremo dai pezzi più facili, i colori e i comandi, e saliremo piano piano.*

Changelog dell'eserciziario

| Versione | Data | Cosa e' cambiato |
|----------|------------|--|
| 0.1 | 26/07/2026 | Prima stesura: Es.1 bottone/Caption, ponte da Lazarus; Es.2 game loop, muovi il quadrato; Es.3 prendi la moneta, movimento+caduta+punteggio. Formato a 4 livelli di aiuto. |
| 0.2 | 26/07/2026 | Introdotti i "Progetti BOSS", giochi pronti da personalizzare. Aggiunto l'Esercizio BOSS "Affonda la Bonomi", battaglia navale 3D: apri · gioca · fallo tuo · il codice nel repository. |
| 0.3 | 26/07/2026 | Aggiunto lo screenshot del gioco "Affonda la Bonomi" nell'Esercizio BOSS. |
| 0.4 | 26/07/2026 | Stile: sottotitoli degli esercizi senza parentesi, titolo più sottotitolo centrato; blocchi di codice nero-su-bianco su fondo chiaro per stampare senza sprecare toner. |
| 0.5 | 26/07/2026 | Aspetto più sobrio: rimosse icone/emoji; parentesi tolte da titoli e grassetti; screenshot del BOSS spostato in cima all'esercizio; le frasi tra virgolette non si spezzano a fine riga. |

Quaderno dello Studente (modello)

Parte: Corso Godot / GDScript

Come si usa (semplice)

- Dopo **ogni lezione** aggiungi una pagina “Lezione”.
- Dopo **ogni esercizio/gioco** aggiungi una pagina “Il mio gioco”.
- Metti sempre uno **screenshot** del tuo lavoro: è la parte più bella. 📸
- Alla fine di ogni pagina, prova a **spiegarlo con parole tue**: se lo sai spiegare, l’hai capito davvero.

PAGINA — Lezione (copia questo blocco ogni volta)

Lezione del _//_ — titolo: __

Cosa ho imparato oggi (con parole mie):

Una cosa nuova che non sapevo:

Screenshot / immagine:



(qui lo studente incolla il suo screenshot)

PAGINA — Il mio gioco (copia questo blocco per ogni esercizio)

Gioco: ____ (esercizio n° _)

Cosa fa il mio gioco:

***Cosa ho cambiato per renderlo MIO
(colore, nome, personaggio...):***

Un problema che ho avuto e come l'ho risolto:

Screenshot del gioco:

 (qui lo studente incolla il suo screenshot)

Lo spiego a un amico in 2 righe:

Suggerimento: le immagini mettile in una cartella `immagini/` accanto a questo file, e richiamale come nell'esempio: `![descrizione](immagini/nome.png)`.